

Homework Chapter6

T6.10

(a)

```
#g in a0,h in a1
blt a0, a1, ELSE
add a0, a0, a1
j END
ELSE:
    sub a0, a0, a1
END:
```

(b)

```
# g in a0, h in a1
bge a0, a1, ELSE
addi a1, a1, 1
j END
ELSE:
    slli a1, a1, 1
END:
```

T6.12

思路：主要想清楚两件事：循环应当怎么写、数组应当怎么寻址

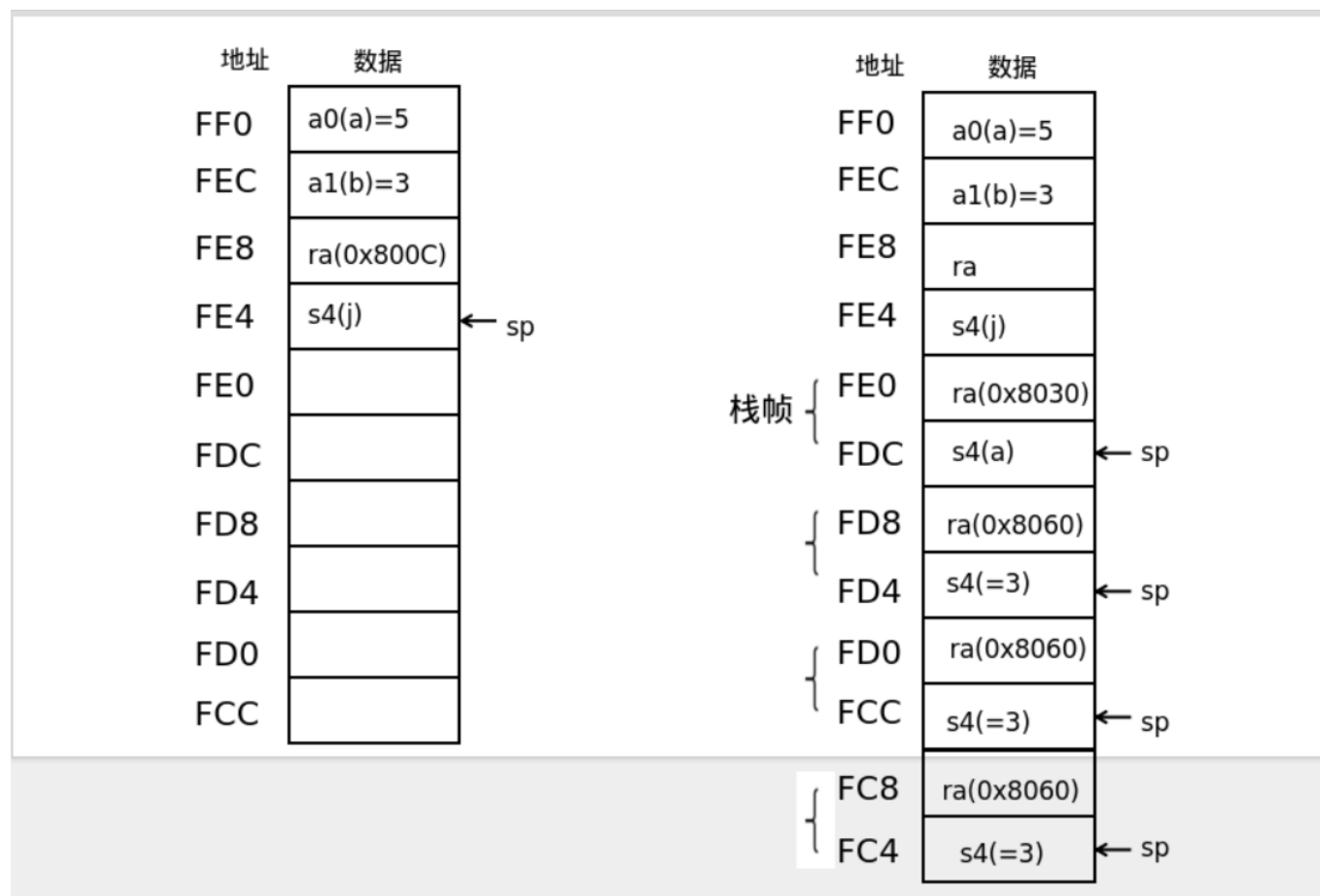
简单描述一下数组寻址的过程：在C语言中， $\&temp[i] = \&temp[0] + i \times 4$ 。由题， $temp[0]$ 存在了t3里面，不妨将*i*存在s0寄存器中，那么每次寻址时应当先计算 $i \times 4$ ，再加上 $\&temp[0]$ (t3)的值，一起得到的就是 $\&temp[i]$ 的地址。注意此时不能更改t3和s0的值，因为s0还要用作循环变量、t3要一直保持是 $\&temp[0]$ 的值，下次寻址的时候还要用。

```

# i in s0, &temp[0] in t3
addi s0, zero, zero
addi t0, zero, 100 # use t0 to represent 100
LOOP:
    bge s0, t0, END
    slli s0, s0, 2 # s0 = i<<2 = i*4
    addi t4, t3, s0 # t4 = &score[i](t3) + i*4(s0)
    lw t5, 0(s0) # t5 = *(t4) = *(&score[i]) = score[i]
    slli t5, t5, 7 # t5 = t5<<7 = t5*128
    sw t5, 0(s0)
    addi s0, s0, 1 # ++i
    j LOOP
END:

```

T6.21



如上图，左侧是调用g函数之前的栈帧状态，右侧是调用g函数时的栈帧状态。

(a) a0的值为19，可以正确计算f(a,b)

(b)程序不会崩溃，但是无法得到正确的结果。0x8014将a的值存在了栈帧上，再次利用是0x8030的指令，此时若改为nop，则0xC(sp)的地址将存放了一段随机数。故得到的结果其实是a+3b+x

(c)

1. 错误结果：f中a0被改为b，由于0x8030行未访存，t0的值未知。实际结果为a+3b+x
2. 崩溃：ra的值被修改，函数找不到正确的返回地址。（不过或许有可能跳到0x8060的位置继续执行，此时ra保存的是g函数最后一次递归的返回地址）
3. 错误结果：没保存j的值，最后只加了一次a。结果为a+3b
4. 错误结果：和1结果一样，不知道t0是什么。
5. 错误结果：g函数内部没有存s4
6. 结果错误：即不保存s4也不恢复s4，只能算出3b+x1+x2
7. 崩溃：g函数内部没保存ra，返回值不知道跳到哪去了。

T6.22

```
addi s3, s4, 28:0x01CA9913
```

```
sll t1, t2, t3:0x01C39333
```

```
srli s3, s1, 14:0x00E4D993
```

```
sw:s9, 16(t4):0x019EA823
```

T6.26

(a)

```
000000010100 00000 000 01010 0010011
```

```
addi a0, zero, 20
```

```
000000000011 00000 000 01011 0010011
```

```
addi a1, zero, 3
```

```
000000000000 00000 000 00111 0010011
```

```
addi t2, zero, 0
```

0000000 00000 01001 000 11100 0110011

add t3, a1, zero

0000000 11100 01010 100 10000 1100011

blt a0, t3, 16

0000000000001 00111 000 00111 0010011

addi t2, t2, 1

0000000 01011 11100 000 11100 0110011

add t3, t3, a1

11111111010111111111 00000 0111111

jal zero, -12 (这个指令有点没明白。是跳到绝对地址-12的位置么?)

0000000 00000 00111 000 01010 0110011

add a0, t2, zero

(b)

```
void f(int a,int b){
    a=20;
    b=3;

    int tmp = b;
    int ans = 0;
    while(tmp <= a){
        tmp += b;
        ++ans;
    }
    return ans;
}
```

(c)功能：该代码返回A/B的结果（向下舍入）

T6.43(b)

```

.file    "T6.43b.c"
.option nopic
.attribute arch, "rv64i2p1_m2p0_a2p1_f2p2_d2p2_c2p0_zicsr2p0"
.attribute unaligned_access, 0
.attribute stack_align, 16
.text
.align   1
.globl   sort
.type    sort, @function
sort:
    addi    sp,sp,-48
    sd      s0,40(sp)
    addi    s0,sp,48
    sd      a0,-40(s0)
    sw      zero,-20(s0)
    j       .L2
.L6:
    sw      zero,-24(s0)
    j       .L3
.L5:
    lw      a5,-24(s0)
    slli    a5,a5,2
    ld      a4,-40(s0)
    add     a5,a4,a5
    lw      a3,0(a5)
    lw      a5,-24(s0)
    addi    a5,a5,1
    slli    a5,a5,2
    ld      a4,-40(s0)
    add     a5,a4,a5
    lw      a5,0(a5)
    mv      a4,a3
    ble     a4,a5,.L4
    lw      a5,-24(s0)
    slli    a5,a5,2
    ld      a4,-40(s0)
    add     a5,a4,a5
    lw      a5,0(a5)
    sw      a5,-28(s0)
    lw      a5,-24(s0)
    addi    a5,a5,1
    slli    a5,a5,2
    ld      a4,-40(s0)
    add     a4,a4,a5
    lw      a5,-24(s0)
    slli    a5,a5,2
    ld      a3,-40(s0)

```

```

    add    a5,a3,a5
    lw     a4,0(a4)
    sw     a4,0(a5)
    lw     a5,-24(s0)
    addi   a5,a5,1
    slli   a5,a5,2
    ld     a4,-40(s0)
    add    a5,a4,a5
    lw     a4,-28(s0)
    sw     a4,0(a5)
.L4:
    lw     a5,-24(s0)
    addiw  a5,a5,1
    sw     a5,-24(s0)
.L3:
    li     a5,9
    lw     a4,-20(s0)
    subw   a5,a5,a4
    sext.w a4,a5
    lw     a5,-24(s0)
    sext.w a5,a5
    blt    a5,a4,.L5
    lw     a5,-20(s0)
    addiw  a5,a5,1
    sw     a5,-20(s0)
.L2:
    lw     a5,-20(s0)
    sext.w a4,a5
    li     a5,8
    ble    a4,a5,.L6
    nop
    nop
    ld     s0,40(sp)
    addi   sp,sp,48
    jr     ra
.size    sort, .-sort
.ident   "GCC: (13.2.0-11ubuntu1+12) 13.2.0"

```

T6.52

关键在于sw s3, 0(s7)这行指令，把0xABCD8765分别写到了100-103四个地址的字节处。再lb s2, 0(s7)，取出地址100的那个字节。若为小端序，则结果是0x65；反之，则为0xAB